

Programación con *sockets* - Interfaces y direcciones

Al escribir un programa C++ que usa *sockets*, con el objetivo de enviar o recibir información, nos veremos obligados a trabajar con direcciones IP. En general, siempre se precisará la **IP del equipo de destino**, con el que se quiere abrir una conexión o al que se pretende enviar un paquete de información (segmento UDP o datagrama IP). En ocasiones, también será preciso contar con la propia IP del equipo que ejecuta el programa, a la que denominamos **IP de origen**.

Dado que existen dos estándares diferentes del protocolo IP (IPv4 e IPv6), que definen **direcciones de distinta longitud y formato**, en la práctica tendremos que usar estructuras de datos específicas para alojar una dirección según su tipo. No obstante, la mayoría de los servicios para trabajar con *sockets* son idénticos para ambas versiones del protocolo IP: crear el *socket*, usarlo para abrir una conexión, enviar o recibir datos, etc. Esta es la razón de que existan otras estructuras de datos, algo más complejas, que sirven para *ocultar* esas diferencias y permitir un uso bastante homogéneo.

Estructuras de datos para almacenar direcciones IP

En esta sección conoceremos las **estructuras de datos** que se emplean para almacenar direcciones IP, tanto IPv4 como IPv6, así como las **funciones** empleadas habitualmente para convertir entre la notación textual, comprensible para las personas, y la representación interna, en la que se almacenan las mencionadas estructuras en memoria.

Representar direcciones IPv4

Como ya sabemos, una dirección IPv4 tiene una longitud de 32 bits. Aunque solemos notar estas direcciones como un cuarteto de números decimales, p.e. **192.168.36.112**, para almacenarlas es suficiente con una variable **entera sin signo de 32 bits** (**unsigned int**). Al examinar el archivo de cabecera **netinet/in.h** encontramos la siguiente definición:

```
typedef uint32_t in_addr_t;
struct in_addr
{
    in_addr_t s_addr;
};
```

La estructura **in_addr** tiene el objetivo de almacenar una dirección IPv4. Solo tiene un miembro: **s_addr**, cuyo tipo es **uint32_t** ya que **in_addr_t** no es más que un pseudónimo para dicho tipo.

Nota

Al igual que otros muchos tipos a medida, **_uint32_t** está definido en el archivo de cabecera **types.h**. En este caso concreto es un pseudónimo para **unsigned int** en procesadores de 32 bits.

Dado que a las personas nos resulta más cómodo recordar y usar direcciones del tipo **192.168.36.112** que un número enorme, en el archivo de cabecera **arpa/inet.h** encontramos las funciones

`inet_ntoa()` e `inet_aton()` para convertir un `struct in_addr` en una cadena de caracteres y viceversa. Introduce en la consola de tu sistema el comando `man inet_ntoa` para acceder a la documentación de estas dos funciones y otras relacionadas.

La mejor vía para entender cómo se usan estas funciones es con un ejemplo como el mostrado en el siguiente programa. Reprodúcelo, compílalo y prueba con distintas direcciones IP como entrada.

```
#include <iostream>
#include <netinet/in.h>
#include <arpa/inet.h>

using namespace std;

int main(int argc, char *argv[]) {
    struct in_addr unaIP;
    unaIP.s_addr = 1234567890;;

    cout << "La IPv4 " << unaIP.s_addr
         << " se nota como " << inet_ntoa(unaIP) << endl;
    if(argc < 2) {
        cout << "Uso: " << argv[0] << " dirección IP" << endl;
        exit(-1);
    }

    struct in_addr dirIPv4;
    if(auto salida = inet_aton(argv[1], &dirIPv4); salida == 1) {
        cout << "La dirección IPv4 " << argv[1]
             << " se representa como " << dirIPv4.s_addr << endl;
    } else {
        cout << "La dirección " << argv[1] << " no parece válida" << endl;
    }
}
```

```
TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE

fcharte@wasp:~/coding_sockets$ g++ -o dirIPv4 dirIPv4.cpp -std=c++2a
fcharte@wasp:~/coding_sockets$ ./dirIPv4 192.168.36.112
La IPv4 1234567890 se nota como 210.2.150.73
La dirección IPv4 192.168.36.112 se representa como 1881450688
fcharte@wasp:~/coding_sockets$
```

Figura 1. El programa muestra la IP indicada en sus dos representaciones.

Representar direcciones IPv6

Las direcciones IPv6 tienen una mayor longitud y, por ello, precisan una estructura de almacenamiento y una notación distintas. La estructura de datos específica para este tipo de direcciones es `in6_addr`. Si examinamos su definición, en el mismo archivo de cabecera `netinet/in.h`, encontraremos lo siguiente:

```

struct in6_addr
{
    union
    {
        uint8_t  __u6_addr8[16];
        uint16_t __u6_addr16[8];
        uint32_t __u6_addr32[4];
    } __in6_u;
    ...
}

```

Dentro de `in6_addr` tenemos una `union` de C que nos permite acceder a los bytes de la dirección IPv6 de **tres formas distintas**: como un vector de 16 bytes, como un vector de 8 enteros de 16 bits o como un vector de 4 enteros de 32 bits, en todos los casos sin signo. Dado que una dirección IPv6 se nota como `2a00:1450:4003:807:12a0:3004:1020:200e`, y que cada grupo de dígitos corresponde a 16 bits, en principio la vía más cómoda para almacenar una dirección de este tipo en una variable de tipo `struct in6_addr` sería la siguiente:

```

struct in6_addr unaIP;

// 2a00:1450:4003:807:12a0:3004:1020:200e
unaIP.__in6_u.__u6_addr16[0] = 0x2a00;
unaIP.__in6_u.__u6_addr16[1] = 0x1450;
...
unaIP.__in6_u.__u6_addr16[7] = 0x200e;

```

Al hacer esto, sin embargo, es probable que terminemos con una dirección IPv6 incorrecta en la variable `unaIP`. De ello dependerá la **arquitectura del microprocesador** de nuestro ordenador. Si es de tipo *big endian* la dirección sería correcta, pero de ser *little endian* no. Esto se debe a que en Internet los datos numéricos se transfieren en formato *big endian* tal y como se explicó en la práctica de introducción a la programación con *sockets*.

Sabiendo esto, la forma correcta de asignar la dirección IPv6 a la variable `unaIP` sería la siguiente:

```

struct in6_addr unaIP;
// 2a00:1450:4003:807:12a0:3004:1020:200e
unaIP.__in6_u.__u6_addr16[0] = htons(0x2a00);
unaIP.__in6_u.__u6_addr16[1] = htons(0x1450);
...
unaIP.__in6_u.__u6_addr16[7] = htons(0x200e);

```

Las funciones `inet_ntoa()` e `inet_aton()`, descritas antes para IPv4, no sirven para convertir direcciones IPv6 del tipo `struct in6_addr` a cadena de caracteres y viceversa. Se implementaron antes de que existiese IPv6. Es la razón de que tengamos a nuestra disposición dos funciones más:

`inet_ntop()` e `inet_pton()`, equivalentes a las anteriores. Su particularidad estriba en que están preparadas para procesar **direcciones tanto IPv4 como IPv6**. Ambas precisan tres parámetros:

- Una constante que denota si la dirección a convertir es IPv4, caso en el que se usa `AF_INET`, o IPv6, representada por `AF_INET6`.
- El dato a convertir, que será una cadena de caracteres en el caso de `inet_pton()` o bien el puntero a una estructura `in_addr/in6_addr` para `inet_ntop()`.
- El destino donde se dejará el resultado de la conversión: la dirección de una estructura `in_addr/in6_addr` para `inet_pton()` y la de una cadena de caracteres para `inet_ntop()`.

El siguiente fragmento de código muestra cómo tomar una dirección IPv6, entregada como parámetro al programa (`argv[1]`), convertirla en una estructura `in6_addr` y mostrar el resultado en la consola. El resultado sería el de la imagen de la Figura 2.

```
...
struct in6_addr dirIPv6;
if(auto salida=inet_pton(AF_INET6, argv[1], &dirIPv6); salida==1) {
    cout << "La dirección IPv6 " << argv[1] << " se almacena como "
        << ntohl(dirIPv6.__in6_u.__u6_addr32[0]) << " "
        << ntohl(dirIPv6.__in6_u.__u6_addr32[1]) << " "
        << ntohl(dirIPv6.__in6_u.__u6_addr32[2]) << " "
        << ntohl(dirIPv6.__in6_u.__u6_addr32[3]) << " "
        << endl;
} else {
    cout << "La dirección " << argv[1] << " no parece válida" << endl;
}
...
```

TERMINAL PROBLEMS OUTPUT DEBUG CONSOLE

```
fcharte@wasp:~/coding_sockets$ g++ -o dirIPv6 dirIPv6.cpp -std=c++2a
fcharte@wasp:~/coding_sockets$
fcharte@wasp:~/coding_sockets$ ./dirIPv6 fe80::215:5dff:fe26:df2e
La dirección IPv6 fe80::215:5dff:fe26:df2e se almacena como 4269801472 0 0 0
fcharte@wasp:~/coding_sockets$
```

Figura 2. El programa convierte una cadena con una IPv6 a su representación interna.

EJERCICIO 1

Modifica el programa que convierte direcciones IPv4, mostrado unas páginas antes, para que use las funciones `inet_ntop()` e `inet_pton()` en lugar de las antiguas `inet_ntoa()` e `inet_aton()`. El resultado generado por el programa debería seguir siendo el mismo. **Detalla a continuación** qué líneas del programa has cambiado (código original) y de qué forma (nuevo código).

Las direcciones IP no viajan solas

Una dirección IP raramente se usa de manera aislada, sino que se acompaña de otros datos como, por ejemplo, el número de puerto. Por ello las estructuras `in_addr` e `in_addr6` que ya conocemos no suelen usarse tal cual, sino como **parte de otras estructuras que las contienen**. Estas son `sockaddr_in` y `sockaddr_in6`. Como es fácil suponer, la primera se asocia a direcciones IPv4 y la segunda a IPv6. Ambas cuentan con los siguientes campos:

- `sin_family`: de tipo `sa_family_t`, almacenará el valor `AF_INET` o `AF_INET6` según el tipo de dirección.
- `sin_port/sin6_port`: de tipo `in_port_t`, contendrá el número de puerto a usar conjuntamente con la IP. El número de puerto ha de facilitarse en representación *big endian*.
- `sin_addr/sin6_addr`: de tipo `in_addr/in6_addr`, servirá para contener la dirección IP.

Así, si quisiéramos preparar la información necesaria para conectar con un servidor HTTP cuya dirección IP es `185.199.108.153`, lo haríamos de la siguiente forma:

```
struct sockaddr_in conf;

memset(&conf, 0, sizeof(conf));
conf.sin_family = AF_INET;
conf.sin_port = htons(80);
inet_pton(conf.sin_family, "185.199.108.153", &conf.sin_addr);

cout << "Configuración de conexión preparada" << endl;
```

De forma análoga se prepararía una estructura `sockaddr_in6` con los datos de una conexión IPv6.

Nota

La función `memset()` se usa para rellenar un bloque de memoria con un cierto valor. En este caso se ponen a 0 todos los bytes que ocupa la estructura almacenada en la variable `conf`. Esta es casi siempre una buena práctica en C++, ya que este lenguaje no inicializa el espacio ocupado por las variables. Un método alternativo es inicializar la variable en el momento de la declaración, como se muestra en el siguiente fragmento de código, lo cual nos ahorra el uso de `memset()` ya que se inicializan a cero todos aquellos miembros de la estructura para los que no se facilita explícitamente un valor:

```
struct sockaddr_in conf = {
    .sin_family = AF_INET,
    .sin_port = htons(80)
};
```

Unificación de direcciones IP

Al modo de las *matrioshka*, las estructuras de datos se encapsulan unas dentro de otras para acoger toda la información necesaria. Una estructura `in_addr` pasa a ser miembro de la estructura `sockaddr_in` y esta, a su vez, se introducirá en una estructura de tipo `addrinfo`. Esta se encuentra definida de la siguiente forma en el archivo de cabecera `netdb.h`:

```
struct addrinfo
{
    int ai_flags;           /* Input flags. */
    int ai_family;         /* Protocol family for socket. */
    int ai_socktype;       /* Socket type. */
    int ai_protocol;       /* Protocol for socket. */
    socklen_t ai_addrlen;   /* Length of socket address. */
    struct sockaddr *ai_addr; /* Socket address for socket. */
    char *ai_canonname;     /* Canonical name for service location. */
    struct addrinfo *ai_next; /* Pointer to next in list. */
};
```

La finalidad de esta estructura es tratar de manera unificada direcciones IP, **sin que importe la versión**, así como aunar otros detalles relativos a una conexión por *sockets*, como es el tipo de *socket*, protocolo de transporte, etc. De los campos que conforman `addrinfo`, y que consultaremos en la correspondiente documentación, son de especial interés los siguientes:

- `ai_addr`: apunta a una estructura `sockaddr_in` o `sockaddr_in6`, según que el campo `ai_family` sea `AF_INET` o `AF_INET6`, respectivamente. En consecuencia, para acceder a esa información es preciso realizar la conversión apropiada.
- `ai_next`: es un puntero a otra estructura del mismo tipo. Sirve para crear listas enlazadas de direcciones, lo que permite mantener unidas múltiples IP, por ejemplo una IPv4 y una IPv6, asociadas a una misma interfaz de red.

El diagrama siguiente resume cómo, partiendo de una variable de tipo `struct addrinfo *`, se determina si la dirección que contiene es IPv4 o IPv6 para, en consecuencia, convertir el campo `ai_addr` al tipo adecuado y recuperar dicha dirección.

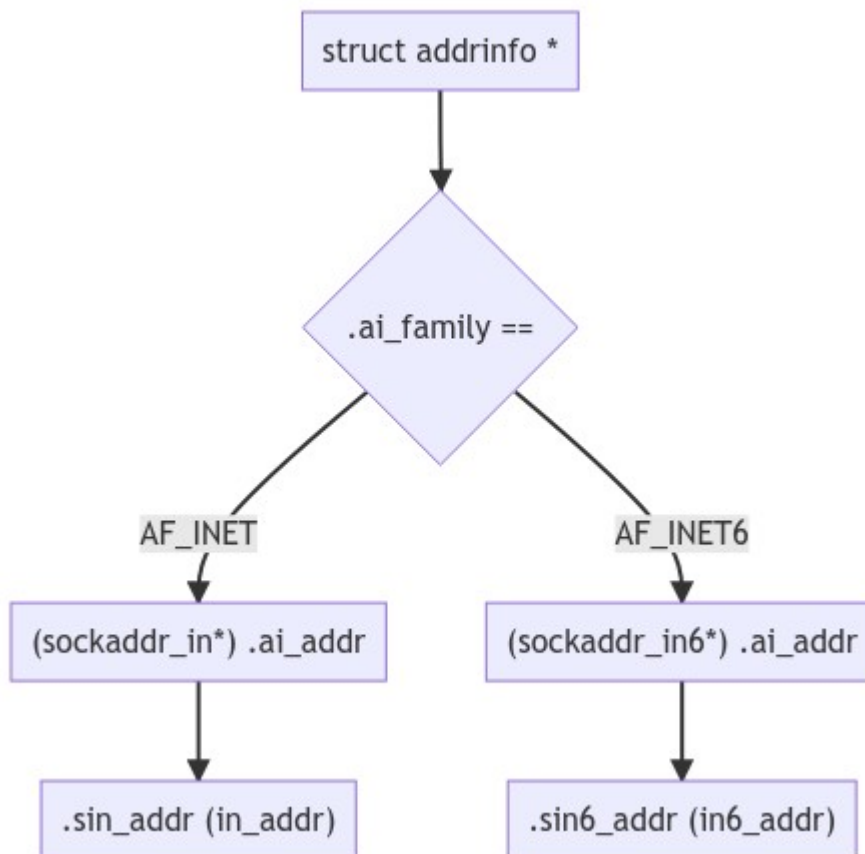


Diagrama 1. Secuencia de comprobaciones y conversiones para extraer la dirección IPv4/IPv6 desde una estructura de tipo `addrinfo`.

EJERCICIO 2

Usa el comando `man getaddrinfo` para acceder a la documentación de dicha función. Al inicio de la misma encontrarás los detalles relativos a la estructura de datos `addrinfo`. Atendiendo a la misma, responde a las siguientes cuestiones:

1. Además de `AF_INET` o `AF_INET6`, ¿qué otro valor puede tomar el campo `ai_family` y cuál es su significado?
2. ¿Qué valores puede tomar el campo `ai_socktype` y qué indican?
3. ¿Cuál es la finalidad del campo `ai_protocol`?

Nota

Además de las ya citadas, en el archivo de cabecera `socket.h` también se encuentra definida la estructura `struct sockaddr_storage`. Esta suele utilizarse con la función `accept()` que conoceremos en una práctica posterior, a fin de recibir conexiones de clientes que pueden usar tanto IPv4 como IPv6.

Resolución de direcciones IP

Normalmente no tendremos que asignar de forma manual el contenido de una estructura `addrinfo`, sino que esta nos vendrá dada como resultado de una **operación de resolución o búsqueda**. Con este

fin siempre usaremos la función `getaddrinfo()`, aquella sobre la que has consultado su documentación en el ejercicio previo.

Nota

En un ejercicio posterior aprenderemos a usar `getaddrinfo()` para obtener direcciones IP de un *host* y mostrarlas en la consola. A continuación se describen los aspectos esenciales de la función. En su página de manual encontramos información más detallada.

Cómo invocar a `getaddrinfo()`

Para llamar a esta función es preciso entregar varios parámetros como entrada, si bien algunos de ellos pueden ser `NULL` en ciertas situaciones. Estos argumentos son:

- Un puntero `char *` con el **nombre del host** cuya dirección se quiere obtener. Puede ser `NULL` si se quiere preparar un *socket* para operar como servidor y quedar a la escucha.
- Otro puntero `char *`, en este caso indicando el **nombre del servicio** (p.e. "`http`") o número de puerto (p.e. "`443`") al que se quiere acceder. También puede ser `NULL` si no quiere seleccionarse un servicio concreto.
- Un `struct addrinfo *` que servirá para **establecer la configuración** que se desea para acceder al *host* indicado: versión de IP a usar, tipo de *socket*, protocolo (TCP/UDP), etc.
- Un `struct addrinfo **` en el que se devolverá la dirección de la primera `struct addrinfo` con los datos recuperados. Esta puede estar **enlazada con otras** (en caso de que se hayan obtenido varias direcciones), por lo que se comprobará el campo `ai_next` y, de no ser `NULL`, se obtendrá la siguiente entrada.

La función devolverá el valor `0` si todo ha ido bien o un código de error en caso contrario, de forma que el resultado almacenado en el último parámetro no será válido.

Obtener direcciones IP

El programa siguiente toma como parámetro el nombre de un servidor en Internet y muestra en la consola su dirección IP. Compíllalo y ejecútalo con `obtenip google.com`, por ejemplo:

```
#include <iostream>
#include <netdb.h>
#include <arpa/inet.h>
using namespace std;

void muestraIP(struct addrinfo *direccion) {
    char strdireccion[NI_MAXHOST + 1];

    inet_ntop(direccion->ai_family,
              direccion->ai_family == AF_INET ?
                  (void *)&((struct sockaddr_in *) direccion->ai_addr)->sin_addr :
                  (void *)&((struct sockaddr_in6 *) direccion->ai_addr)->sin6_addr,
              strdireccion, NI_MAXHOST);

    cout << strdireccion << endl;
}
```



```
int main(int argc, char *argv[]) {
    struct addrinfo configuracion, *resultado;

    configuracion = {};
    configuracion.ai_family = AF_UNSPEC;
    configuracion.ai_socktype = SOCK_STREAM;

    getaddrinfo(argv[1], NULL, &configuracion, &resultado);

    muestraIP(resultado);
    freeaddrinfo(resultado);
}
```

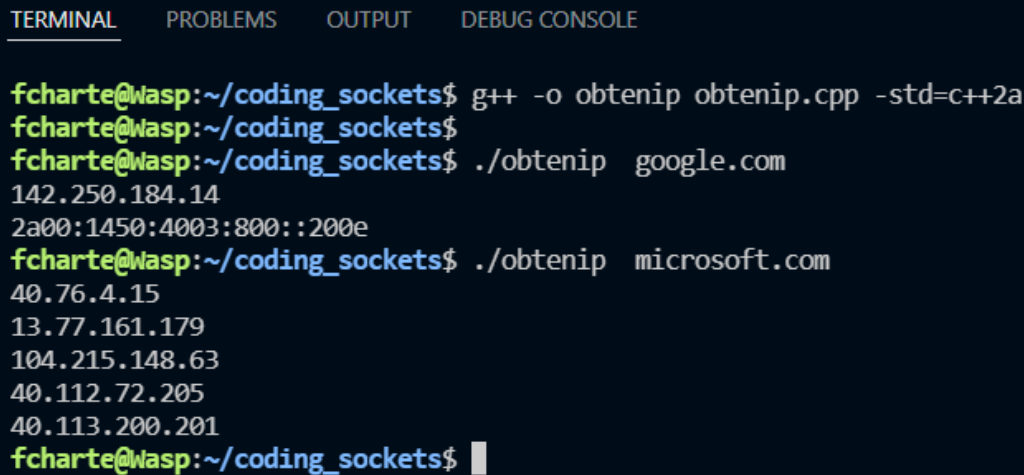
La función `muestraIP()` recibe como argumento un `struct addrinfo *`, comprueba qué valor contiene su campo `ai_family` para determinar si la dirección es IPv4 o IPv6 y, en consecuencia, convierte el campo `ai_addr` al tipo adecuado: `sockaddr_in` o `sockaddr_in6`. Se obtiene la dirección de ese puntero y se convierte a `void *` para entregarlo como entrada a la función `inet_ntop()` que ya conocemos. Por último, muestra por la consola la dirección tras la conversión.

En la función `main()` se prepara una estructura `addrinfo` como configuración de entrada a la consulta que efectuará `getaddrinfo()`, indicando que nos da igual el tipo de dirección (`AF_UNSPEC`) y que la usaremos para trabajar con un `socket` de tipo `SOCK_STREAM`. El resultado de la consulta se almacena en la variable `resultado`, entregada después como parámetro a `muestraIP()`. La última sentencia se asegura de liberar la memoria asignada por la invocación a `getaddrinfo()`.

EJERCICIO 3

Tomando el programa anterior como punto de partida, realiza las modificaciones indicadas a continuación:

1. Comprueba que al invocar al programa se ha facilitado el parámetro necesario. En caso de no ser así, avisa al usuario con un **mensaje indicativo del modo de uso**, como haría cualquier utilidad invocada desde la línea de comandos.
2. Verifica que la llamada a `getaddrinfo()` **no devuelve un código de error** antes de usar el valor alojado en `resultado`. Genera una excepción en caso de error para que el programa no continúe la ejecución.
3. Introduce la sentencia `muestraIP(resultado);` en un bucle que recorra todas las direcciones devueltas por `getaddrinfo()`. Para ello, toma el contenido del campo `ai_next` y, si no es `NULL`, úsalo para avanzar en la iteración de la lista enlazada. El funcionamiento del programa debería ser similar al mostrado en la siguiente imagen siguiente.



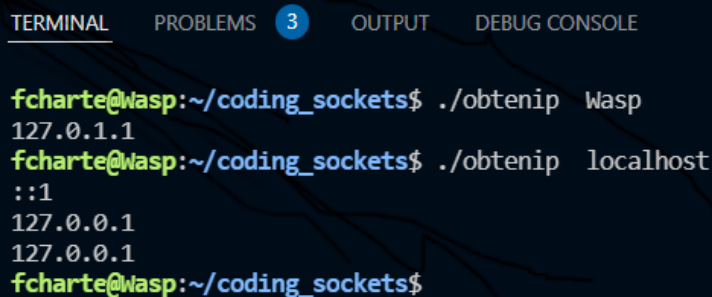
```
TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE

fcharte@wasp:~/coding_sockets$ g++ -o obtenip obtenip.cpp -std=c++2a
fcharte@wasp:~/coding_sockets$
fcharte@wasp:~/coding_sockets$ ./obtenip google.com
142.250.184.14
2a00:1450:4003:800::200e
fcharte@wasp:~/coding_sockets$ ./obtenip microsoft.com
40.76.4.15
13.77.161.179
104.215.148.63
40.112.72.205
40.113.200.201
fcharte@wasp:~/coding_sockets$
```

Figura 3. El programa muestra todas las IP asociadas al *host*.

Cómo obtener la IP propia

Con la función `getaddrinfo()` es posible resolver direcciones IP de *hosts* a partir de su nombre, pero **no nos servirá** si lo que queremos es conocer las direcciones IP de nuestras propias interfaces de red. Puedes comprobarlo ejecutando el anterior programa facilitándole como parámetro `localhost` o el nombre que tenga asignado tu equipo. La respuesta siempre es la dirección local, no la IP con la que nos conectamos a nuestra red local y, desde esta, a Internet.



```
TERMINAL  PROBLEMS  3  OUTPUT  DEBUG CONSOLE

fcharte@wasp:~/coding_sockets$ ./obtenip wasp
127.0.1.1
fcharte@wasp:~/coding_sockets$ ./obtenip localhost
::1
127.0.0.1
127.0.0.1
fcharte@wasp:~/coding_sockets$
```

Figura 4. El programa devuelve la dirección del bucle local o *loopback*, no la IP asignada a la interfaz de red con la que estamos conectados a nuestra red local.

Para obtener información sobre las interfaces de red de nuestro equipo, incluyendo las direcciones IPv4 e IPv6 que tuvieran asignadas, la función a emplear es `getifaddrs()`. Esta devuelve una lista enlazada de tipo `struct ifaddrs`. Para una misma interfaz, cuyo nombre se incluye en la estructura, se devolverán varios elementos: uno con la dirección MAC (capa de enlace), otro con la IPv4 y, si procede, otro con la IPv6. Esta función también facilita otros datos, como la máscara de red, paquetes enviados y recibidos, etc.

Aviso

Algunos de los campos de esta estructura son válidos solo si están activos ciertos indicadores, por lo que acceder a ellos sin una verificación previa puede producir errores.

La información devuelta por `getifaddrs()` se almacena en un bloque de memoria asignado dinámicamente, por lo que una vez no nos sea útil hemos de liberarlo mediante la función `freeifaddrs()`.

EJERCICIO 4

Consulta con `man getifaddrs` la documentación relativa a esta función. En ella encontrarás los detalles relativos al tipo `struct ifaddrs` y el contenido de cada uno de sus campos. Con esa información, escribe un programa que muestre por consola datos sobre las interfaces de red de tu equipo.

Comienza por una versión simple, que solo enumere el nombre de las interfaces, y después, a medida que leas la documentación de cada campo de `struct ifaddrs`, mejora el programa para que amplíe la información facilitada **hasta donde te sea posible**. La imagen siguiente muestra cómo podría ser la salida de dicho programa:

```
fcharte@Wasp:~/coding_sockets$ ./lista_interfaces
eth0
    inet 172.19.244.220 - netmask 255.255.240.0
    inet6 fe80::215:5dff:fe26:df2e%eth0 - netmask ffff:ffff:ffff:ffff::
    link 00:15:5D:26:DF:2E
    TX: 9580 paquetes, 806185 bytes
    RX: 68687 paquetes, 124257189 bytes
lo
    inet 127.0.0.1 - netmask 255.0.0.0
    inet6 ::1 - netmask ffff:ffff:ffff:ffff:ffff:ffff:ffff:ffff
    link 00:00:00:00:00:00
    TX: 879200 paquetes, 1034396362 bytes
    RX: 879200 paquetes, 1034396362 bytes
```

Figura 5. El programa facilita por cada interfaz de red su nombre, direcciones IP y MAC, paquetes y bytes recibidos y transmitidos.

Material complementario

Los modificadores de flujo de la biblioteca `io manip` de C++ son herramientas muy útiles para dar formato a la salida por consola, ayudando a crear una presentación más clara de los datos.

¿Qué son los modificadores de flujo?

Son funciones que se aplican a los objetos de flujo de salida (como `cout`) para modificar el comportamiento de las operaciones de salida. Estos modificadores te permiten controlar aspectos como:

- **Ancho de campo:** el número mínimo de caracteres que ocupará un valor en la salida.
- **Alineación:** indica cómo se alineará el valor a escribir, a la izquierda, derecha o al centro dentro del campo.
- **Relleno:** establece el carácter que se utilizará para rellenar el espacio sobrante en el campo.
- **Precisión:** fija el número de dígitos significativos para números de punto flotante.

- **Base numérica:** cambia la base para mostrar números entre decimal, hexadecimal, octal, etc.

Modificadores más comunes

La lista siguiente enumera los modificadores de uso más común:

- `setw(n)`: establece el ancho de campo en `n` caracteres.
- `setfill(c)`: establece `c` como carácter de relleno.
- `left`: alinea el dato a la izquierda.
- `right`: alinea el dato a la derecha.
- `setprecision(n)`: establece la precisión para números de punto flotante.
- `fixed`: muestra números de punto flotante en notación decimal fija.
- `scientific`: muestra números de punto flotante en notación científica.

Ejemplo de uso de los modificadores

El siguiente código de ejemplo demuestra cómo emplear los modificadores indicados para generar una salida en forma de tabla a partir de datos almacenados en varios vectores, uno con cadenas y otro con números:

```
#include <iostream>
#include <iomanip>
#include <string>

using namespace std;

int main() {
    string productos[] = {"Manzana", "Banana", "Naranja", "Pera"};
    double precios[] = {0.99, 0.7, 1.253, 0.89};
    int stock[] = {100, 150, 80, 120};

    // Encabezado
    cout << left << setw(15) << "Producto"
         << setw(10) << "Precio"
         << right << setw(10) << "Stock" << endl;
    cout << setfill('-') << setw(35) << "" << endl << setfill(' ');

    // Formato para números con dos decimales y alineación a la derecha
    cout << fixed << setprecision(2);

    // Imprimir los datos
    for (int i = 0; i < 4; ++i) {
        cout << left << setw(15) << productos[i]
             << right << setw(7) << precios[i]
             << setw(13) << stock[i] << endl;
    }

    return 0;
}
```

```
francisco@ANTMAN:Francisco$ g++ formato.cpp -o formato
francisco@ANTMAN:Francisco$
francisco@ANTMAN:Francisco$ ./formato
Producto          Precio          Stock
-----
Manzana           0.99           100
Banana            0.70           150
Naranja           1.25            80
Pera              0.89           120
francisco@ANTMAN:Francisco$
francisco@ANTMAN:Francisco$ |
```

Figura 6. Salida que genera el programa, en forma de tabla, gracias al uso de los manipuladores de `iomanip`.